TÓPICO 5

CSS – Layouts com Flexbox e Grid



Prof. Fábio Ferreira
fabio.ferreira@ifsudestemg.edu.br

Nos tópicos anteriores, aprendemos a utilizar o **CSS** para controlar a apresentação visual dos elementos **HTML**, definindo cores, fontes, bordas, espaçamentos e dimensões. No entanto, construir páginas **Web** também exige organizar esses elementos na página, definindo seu alinhamento, distribuição e posicionamento.

O **CSS** oferece diferentes recursos para a construção de layouts. Entre eles, destacam-se o **Flexbox** e o **Grid Layout**, que permitem organizar elementos de maneira flexível e estruturar conteúdos em linhas e colunas. Esses recursos facilitam a criação de interfaces organizadas e adaptáveis a diferentes necessidades.

Neste tópico, vamos conhecer os principais conceitos relacionados ao Flexbox e ao Grid Layout, explorando suas propriedades e aplicações por meio de exemplos práticos. Ao longo do estudo, esses recursos serão utilizados para organizar os elementos de páginas **Web**, aplicando os conhecimentos ao conteúdo dos tópicos anteriores.

Ao término deste tópico, você será capaz de:

- ✓ Compreender os principais conceitos relacionados à construção de layouts com **CSS**;
- ✓ Utilizar **Flexbox** para alinhar e distribuir elementos em um contêiner;
- ✓ Utilizar **Grid Layout** para organizar conteúdos em linhas e colunas;
- ✓ Aplicar Flexbox e Grid Layout na construção de layouts para páginas **Web**.

1. Introdução aos layouts CSS

Ao desenvolver uma página **Web**, não basta definir o conteúdo **HTML** e sua aparência visual. Também precisamos organizar os elementos na página, controlando como eles são distribuídos, alinhados e dimensionados em relação aos demais elementos. Essa organização é chamada de **layout**. Um layout define, por exemplo, como os itens de um menu são distribuídos horizontalmente, como os conteúdos de uma página são organizados em colunas ou como os elementos de um formulário são alinhados.

O **CSS** oferece diferentes recursos para construir layouts. Entre eles, destacam-se o **Flexbox** e o **Grid Layout**, que permitem controlar a organização dos elementos de maneira mais flexível do que o fluxo normal do documento **HTML**. O **Flexbox** é especialmente útil quando precisamos organizar elementos em uma única dimensão, seja em uma linha ou em uma coluna. Já o **Grid Layout** permite organizar os elementos em duas dimensões, controlando simultaneamente linhas e colunas.

A **Figura 1** apresenta três formas de organizar os elementos de uma página **Web**: o fluxo normal do **HTML**, no qual os elementos são exibidos sequencialmente; o Flexbox, que permite organizá-los em uma linha ou coluna; e o Grid Layout, que possibilita distribuir os elementos em linhas e colunas. Essa comparação ajuda a compreender as diferenças entre essas abordagens. Ao longo deste tópico, vamos explorar esses dois recursos e compreender como utilizá-los na construção de layouts para páginas **Web**.

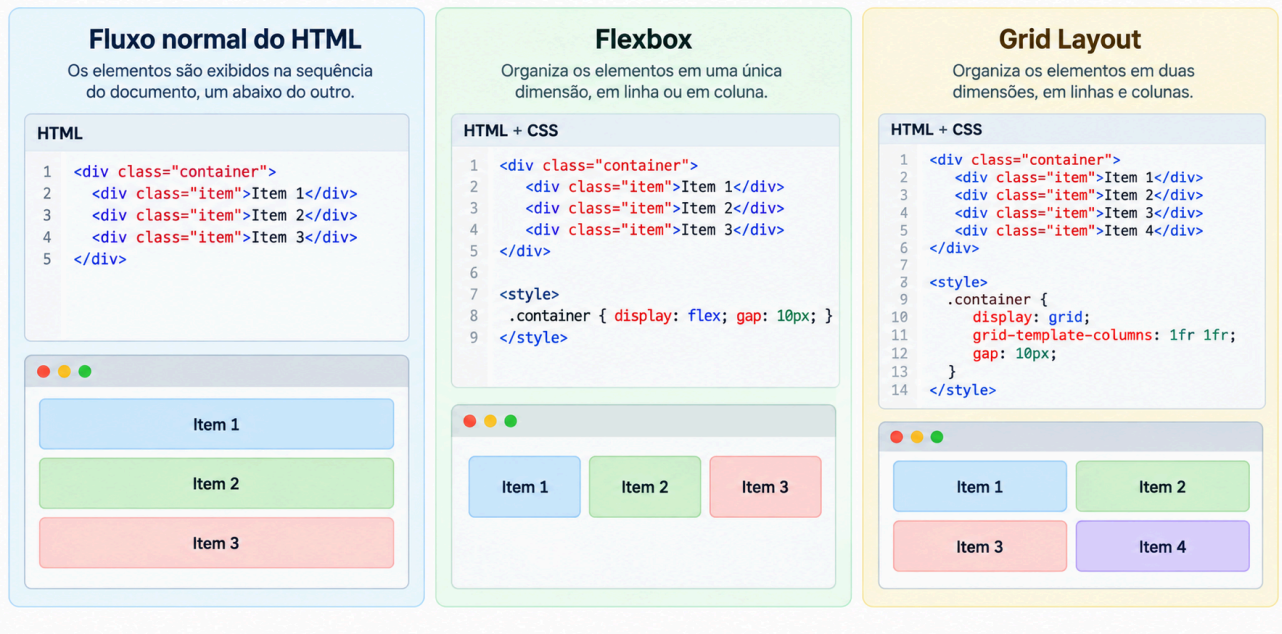


Figura 1 – Comparação entre o fluxo normal do HTML, Flexbox e Grid Layout.

2. Flexbox

O **Flexbox** (Flexible Box Layout) é um recurso do **CSS** que permite organizar e distribuir elementos dentro de um contêiner, facilitando o alinhamento e o controle dos espaços entre eles. É especialmente útil quando precisamos posicionar elementos em uma única dimensão, isto é, em uma linha ou em uma coluna.

Para utilizar o Flexbox, definimos a propriedade `display: flex` no elemento que contém os itens que desejamos organizar. Esse elemento passa a ser chamado de **contêiner flexível**, enquanto seus elementos filhos diretos são chamados de **itens flexíveis**. Por exemplo, considere o **HTML** a seguir:

```HTML
<div class="container">
    <div>Item 1</div>
    <div>Item 2</div>
    <div>Item 3</div>
</div>
```

Podemos utilizar o **CSS** para transformar o contêiner em um contêiner flexível:

```CSS
.container {
    display: flex;
}
```

Por padrão, os itens flexíveis são organizados em uma linha, da esquerda para a direita, conforme a direção de escrita habitual. Dessa forma, os três elementos do exemplo são apresentados lado a lado.

A **Figura 2** compara a disposição dos elementos **HTML** no fluxo normal do documento com a organização obtida ao utilizar o Flexbox. Como podemos observar, sem essa propriedade os elementos são apresentados um abaixo do outro, enquanto `display: flex` permite organizá-los lado a lado, facilitando a construção de layouts.

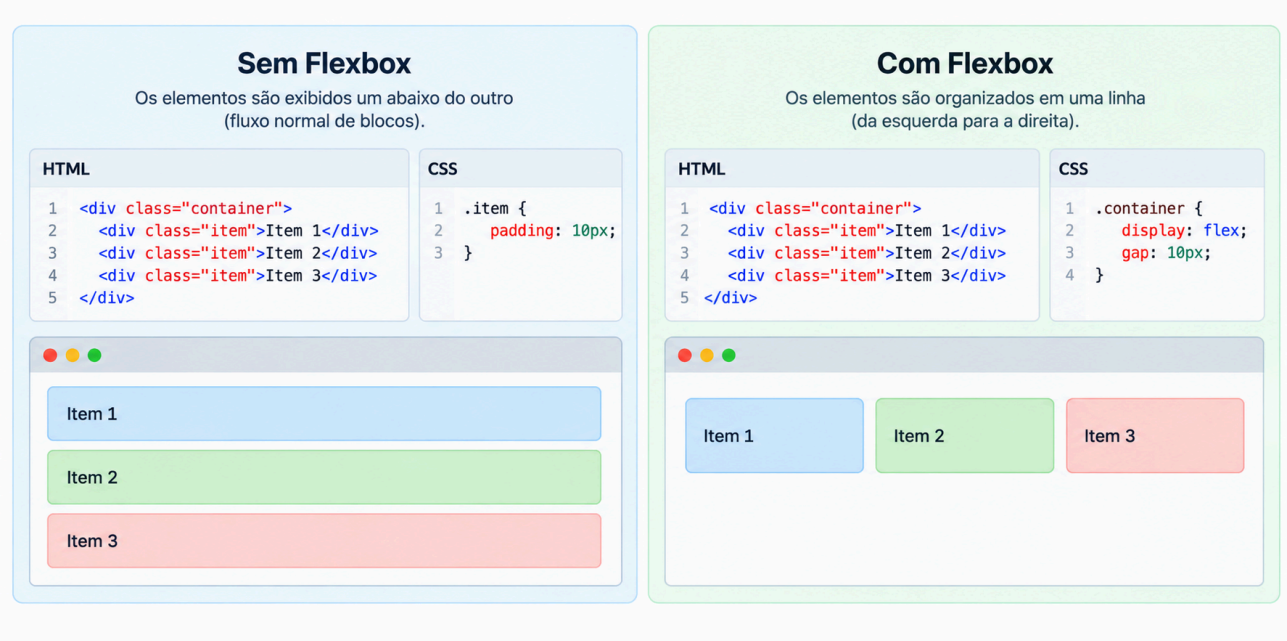


Figura 2 – Alteração da disposição dos elementos com display: flex.

Além de `display: flex`, o **CSS** oferece também o valor `display: inline-flex`, que também transforma um elemento em um contêiner Flexbox. A diferença está no comportamento do próprio contêiner em relação aos demais elementos da página. Com `display: flex`, o contêiner se comporta como um elemento de bloco, ocupando, por padrão, toda a largura disponível. Já com `display: inline-flex`, ele se comporta como um elemento de nível de linha, ocupando o espaço necessário ao seu conteúdo e podendo aparecer ao lado de outros elementos.

Nos dois casos, os elementos filhos diretos são organizados de acordo com as propriedades do Flexbox. Portanto, a diferença está no comportamento externo do contêiner, e não na forma como seus itens são organizados internamente.

A **Figura 3** compara o comportamento das propriedades `display: flex` e `display: inline-flex`. Em ambos os casos, os elementos internos são organizados utilizando o Flexbox. A diferença está no comportamento do próprio contêiner: enquanto `flex` ocupa, por padrão, toda a largura disponível, `inline-flex` ocupa apenas o espaço necessário ao seu conteúdo, permitindo que outros elementos apareçam na mesma linha, quando houver espaço suficiente.

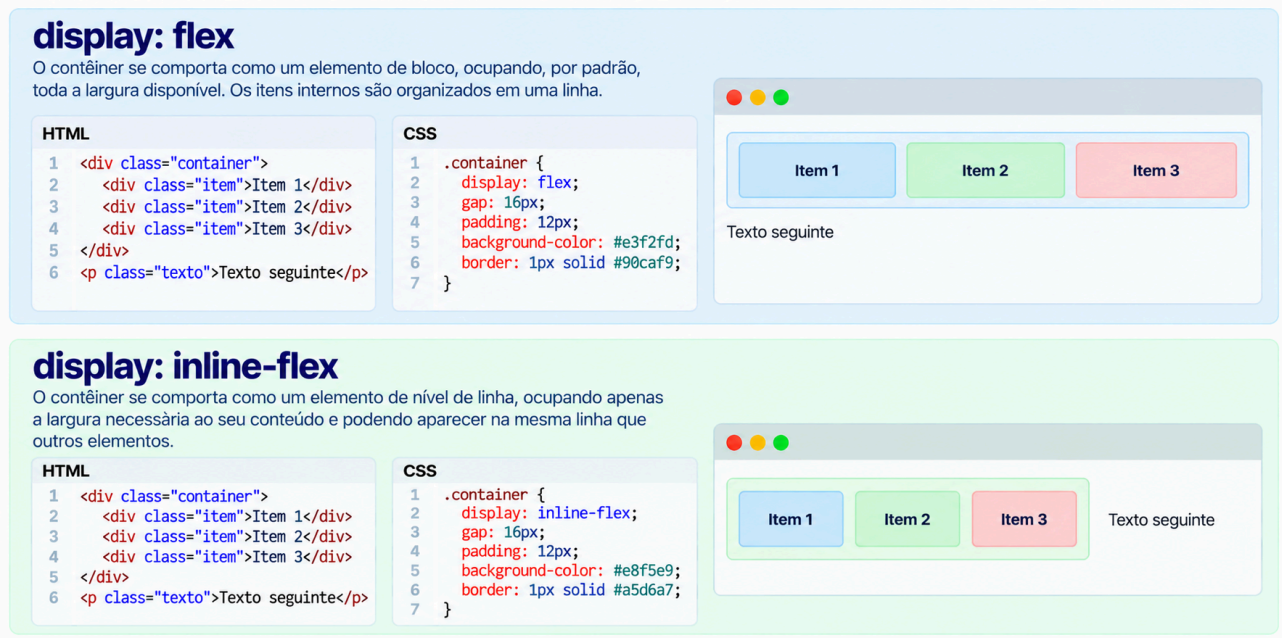


Figura 3 – Comparação entre display: flex e display: inline-flex.

O Flexbox oferece outras propriedades para controlar a direção, o alinhamento e a distribuição dos itens. Nas próximas seções, vamos conhecer as principais.

2.1. Direção dos itens com flex-direction

Por padrão, o Flexbox organiza os itens em uma linha, da esquerda para a direita. No entanto, podemos alterar essa disposição utilizando a propriedade `flex-direction`, que define a direção do eixo principal do contêiner flexível.

Essa propriedade aceita diferentes valores:

- `row`: organiza os itens em uma linha, da esquerda para a direita. É o valor padrão.
- `row-reverse`: organiza os itens em uma linha, invertendo a ordem visual.
- `column`: organiza os itens em uma coluna, de cima para baixo.
- `column-reverse`: organiza os itens em uma coluna, invertendo a ordem visual.

Por exemplo, considere o seguinte código **CSS**:

```
CSS
.container {
    display: flex;
    flex-direction: column;
    gap: 10px;
}
```

Nesse caso, os itens serão organizados verticalmente, um abaixo do outro, com um espaço de `10px` entre eles. Se utilizarmos `flex-direction: row`, os itens serão organizados horizontalmente.

A **Figura 4** apresenta os quatro valores possíveis, mostrando como os elementos são organizados horizontal ou verticalmente e como sua ordem é invertida quando utilizamos os valores terminados com o sufixo `-reverse`.

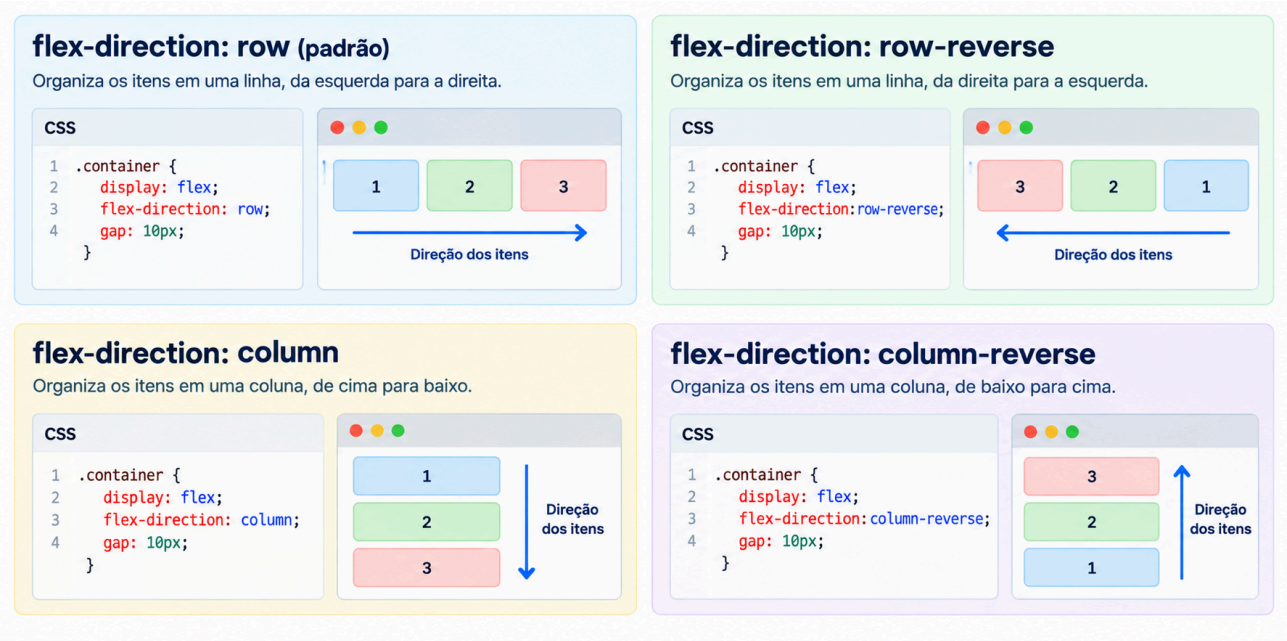


Figura 4 – Diferentes valores da propriedade flex-direction no Flexbox.

A propriedade `flex-direction` permite, portanto, adaptar a posição dos elementos conforme as necessidades do layout.

2.2. Alinhamento e distribuição com justify-content

A propriedade `justify-content` controla como os itens flexíveis são distribuídos ao longo do **eixo principal** do contêiner. Esse eixo depende da direção definida por `flex-direction`: quando os itens estão organizados em uma linha, o eixo principal é horizontal; quando estão organizados em uma coluna, ele é vertical. Essa propriedade aceita diferentes valores, que permitem alinhar os itens no início ou no final do contêiner, centralizá-los ou distribuir o espaço disponível entre eles.

Os valores mais utilizados são:

- `flex-start`: posiciona os itens no início do eixo principal. É o valor padrão.
- `flex-end`: posiciona os itens no final do eixo principal.
- `center`: centraliza os itens ao longo do eixo principal.
- `space-between`: distribui os itens de modo que o espaço disponível fique entre eles, sem espaço adicional nas extremidades.
- `space-around`: distribui o espaço ao redor dos itens, de modo que cada item tenha espaço em ambos os lados.
- `space-evenly`: distribui o espaço livre igualmente entre os itens e as duas extremidades do contêiner.

Por exemplo, podemos centralizar os itens horizontalmente utilizando:

```css
.container {
    display: flex;
    justify-content: center;
    gap: 10px;
}
```

Nesse caso, os itens serão agrupados no centro do contêiner, mantendo um espaço de `10px` entre eles. O `gap` define a distância entre os itens, enquanto `justify-content` controla sua distribuição ao longo do eixo principal.

A **Figura 5** apresenta diferentes valores da propriedade `justify-content`, que controla a distribuição dos itens ao longo do eixo principal de um contêiner Flexbox. Observe como os elementos são posicionados no início, no final ou no centro do contêiner e como o espaço disponível é distribuído entre os itens e as extremidades.

A escolha do valor depende do resultado desejado. Por isso, é importante experimentar diferentes opções e observar como elas alteram a organização dos elementos.

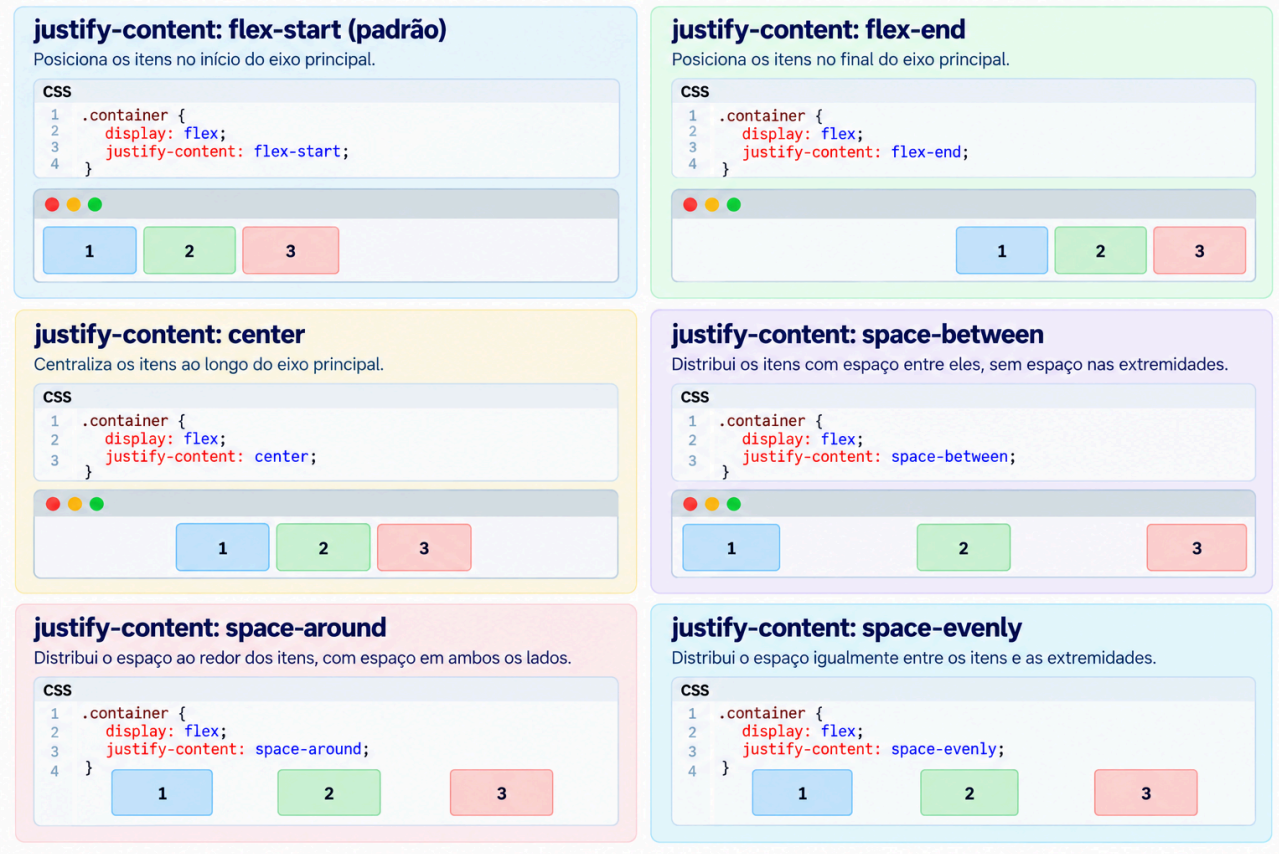


Figura 5 – Diferentes valores da propriedade justify-content no Flexbox.

2.3. Alinhamento dos itens com align-items

Enquanto a propriedade `justify-content` controla a distribuição dos itens ao longo do eixo principal, a propriedade `align-items` controla o alinhamento dos itens no **eixo transversal**, ou seja, no eixo perpendicular ao eixo principal.

Quando utilizamos `flex-direction: row`, o eixo principal é horizontal e o eixo transversal é vertical. Já com `flex-direction: column`, o eixo principal é vertical e o eixo transversal é horizontal.

A propriedade `align-items` aceita diferentes valores, entre os quais se destacam:

- `stretch`: estica os itens para preencher o espaço disponível no eixo transversal, desde que suas dimensões não impeçam esse comportamento. É o valor padrão.
- `flex-start`: alinha os itens no início do eixo transversal.
- `flex-end`: alinha os itens no final do eixo transversal.
- `center`: centraliza os itens no eixo transversal.

Por exemplo, podemos alinhar os itens verticalmente ao centro de um contêiner com direção horizontal:

```css
.container {
    display: flex;
    align-items: center;
}
```

A **Figura 6** apresenta diferentes valores da propriedade `align-items` no Flexbox, mostrando como os itens são alinhados ao longo do eixo transversal do contêiner. Observe que `stretch` estica os itens para preencher o espaço disponível, enquanto `flex-start`, `center` e `flex-end` alteram seu alinhamento dentro do contêiner.



Figura 6 – Diferentes valores da propriedade align-items no Flexbox.

É importante observar que a propriedade `align-items` atua no eixo transversal, cuja orientação depende do valor da propriedade `flex-direction`. Portanto, quando a direção for `column`, o alinhamento controlado por essa propriedade passa a ocorrer horizontalmente.

2.4. Espaçamento entre os itens com gap

A propriedade `gap` permite definir o espaço entre os itens de um contêiner Flexbox. Dessa forma, podemos controlar a distância entre os elementos sem precisar adicionar margens individualmente a cada um deles.

Por exemplo, podemos definir um espaço de `16px` entre os itens:

```css
.container {
    display: flex;
    gap: 16px;
}
```

Nesse caso, haverá um espaço de `16px` entre cada par de itens consecutivos. Esse espaço é aplicado entre os elementos, sem adicionar espaço extra antes do primeiro item ou depois do último.

A propriedade `gap` também pode ser utilizada quando os itens estão organizados em uma coluna:

```css
.container {
    display: flex;
    flex-direction: column;
    gap: 16px;
}
```

Nesse exemplo, os itens serão organizados verticalmente, mantendo um espaço de `16px` entre eles. O `gap` pode ser combinado com propriedades como `justify-content` e `align-items`, permitindo controlar tanto o espaçamento entre os itens quanto sua distribuição e seu alinhamento no contêiner.

2.5. Quebra de linha com flex-wrap

Por padrão, o Flexbox tenta manter todos os itens em uma única linha, mesmo quando o espaço disponível no contêiner é insuficiente. A propriedade `flex-wrap` permite alterar esse comportamento, possibilitando que os itens sejam distribuídos em várias linhas ou colunas, conforme a direção definida. Essa propriedade aceita três valores principais:

- `nowrap`: mantém todos os itens em uma única linha ou coluna. É o valor padrão.
- `wrap`: permite que os itens passem para uma nova linha ou coluna quando não houver espaço suficiente.
- `wrap-reverse`: permite a quebra, mas inverte a direção em que as linhas ou colunas são distribuídas no eixo transversal.

A **Figura 7** apresenta uma comparação entre os três valores da propriedade `flex-wrap`. Observe como `nowrap` mantém todos os itens em uma única linha, enquanto `wrap` permite que os itens passem para uma nova linha quando não há espaço suficiente no

contêiner. Já o valor `wrap-reverse` permite a quebra dos itens, mas inverte a disposição das linhas no eixo transversal.

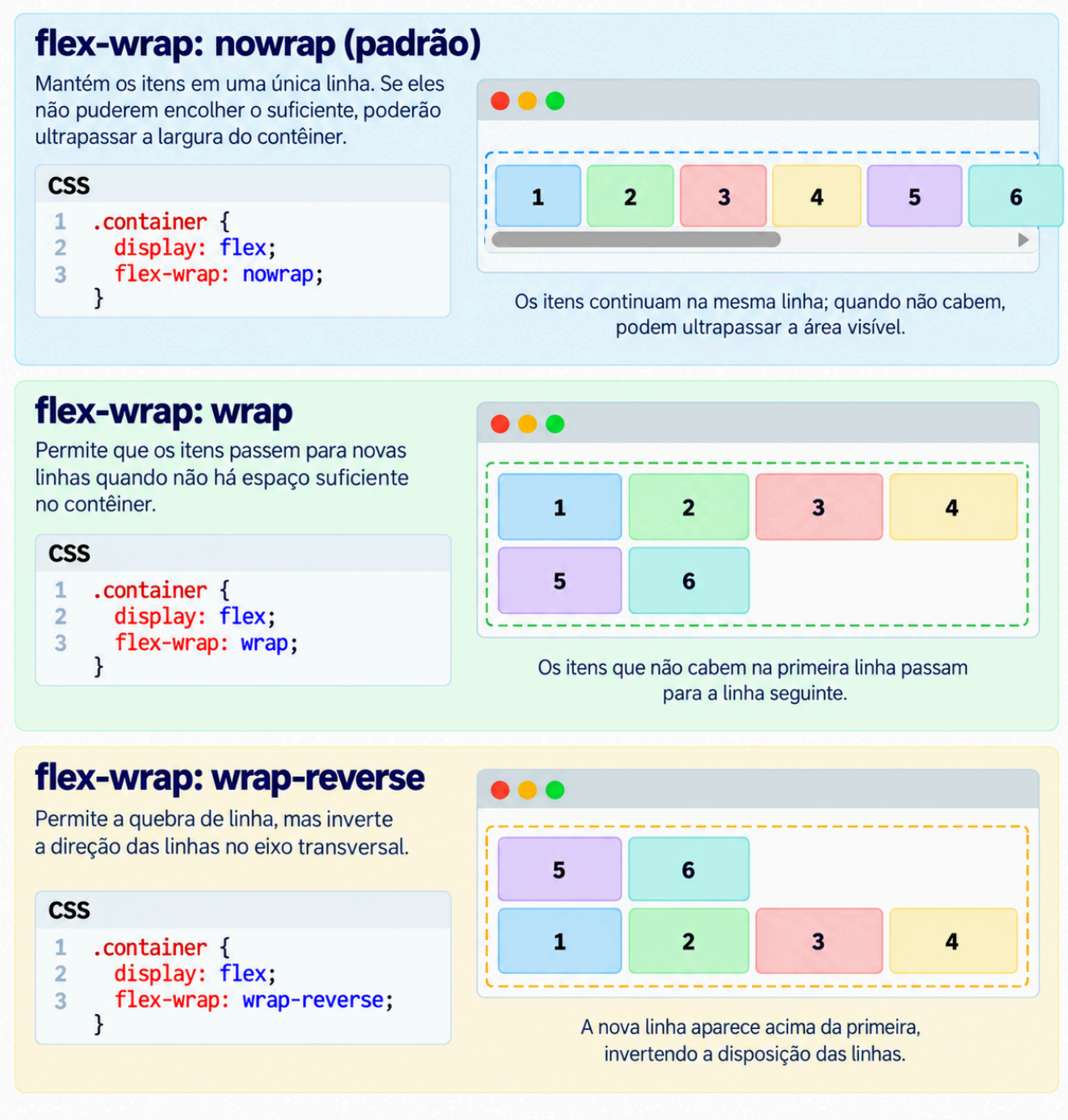


Figura 7 – Comparação dos valores da propriedade flex-wrap no Flexbox.

O valor `wrap` é especialmente útil quando trabalhamos com vários elementos, como cartões, imagens ou opções de navegação, pois permite aproveitar melhor o espaço disponível. É importante lembrar que o `flex-wrap` não altera a direção principal dos itens. Ele apenas permite ou controla a quebra em linhas ou colunas adicionais.

3. Grid Layout

O Flexbox permite organizar elementos **HTML** em uma única dimensão, distribuindo-os em uma linha ou coluna. No entanto, alguns layouts exigem maior controle sobre a organização dos elementos, considerando simultaneamente linhas e colunas. Para essas situações, o **CSS** oferece o **Grid Layout**, um recurso que permite construir layouts bidimensionais.

Com o Grid Layout, podemos definir a quantidade e o tamanho das colunas e linhas de um contêiner, controlar o espaçamento entre os elementos e determinar quais áreas cada item deve ocupar. Esse recurso é especialmente útil na organização de estruturas como galerias de imagens, conjuntos de cartões e diferentes regiões de uma página **Web**.

Embora Flexbox e Grid Layout possam ser utilizados em conjunto, cada recurso apresenta características que o tornam mais adequado a determinadas situações. Enquanto o Flexbox é especialmente útil para distribuir elementos em uma única dimensão, o Grid Layout facilita a organização de estruturas que precisam de controle simultâneo sobre linhas e colunas.

A **Figura 8** compara as duas abordagens para organizar elementos em uma página **Web**. Com o Flexbox, os itens são distribuídos em uma única dimensão, formando uma linha ou coluna. Já o Grid Layout permite organizar os elementos em duas dimensões, controlando simultaneamente linhas e colunas. Essa diferença ajuda a compreender as características de cada recurso e a identificar situações em que podem ser utilizados na construção de layouts.

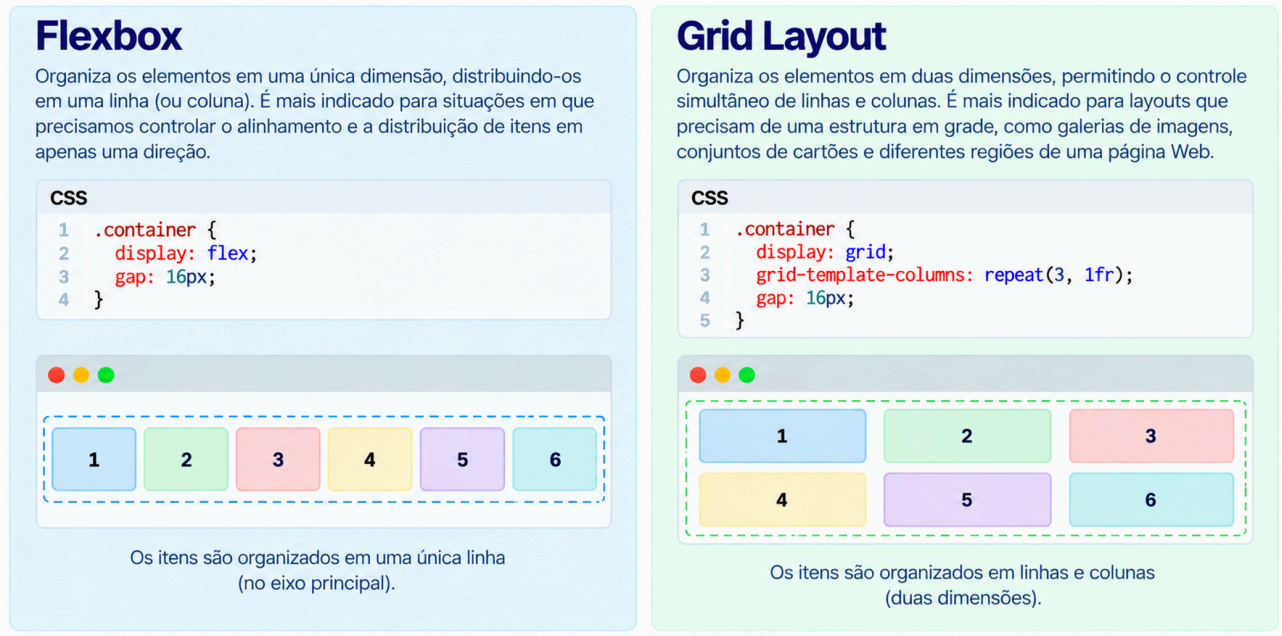


Figura 8 – Comparação entre a organização de elementos com Flexbox e Grid Layout.

Ao longo desta seção, vamos conhecer as principais propriedades do Grid Layout e aprender a utilizá-las na construção de layouts para páginas **Web**.

3.1. Criando um contêiner Grid

O Grid Layout é ativado por meio da propriedade `display`, utilizando o valor `grid`. Quando aplicamos essa propriedade a um elemento **HTML**, ele passa a funcionar como um contêiner Grid, e seus elementos filhos diretos tornam-se itens do Grid. Por exemplo, considere o código **HTML** a seguir:

```HTML
<div class="container">
    <div>Item 1</div>
    <div>Item 2</div>
    <div>Item 3</div>
    <div>Item 4</div>
    <div>Item 5</div>
    <div>Item 6</div>
</div>
```

Para transformar o elemento com a classe `container` em um contêiner Grid, podemos definir a seguinte regra **CSS**:

```CSS
.container {
    display: grid;
}
```

Ao aplicar `display: grid`, os elementos filhos diretos passam a ser organizados de acordo com as regras do Grid Layout. Como ainda não definimos explicitamente as colunas, o navegador utiliza o comportamento padrão do Grid, criando uma única coluna para acomodar os itens.

A **Figura 9** compara a disposição dos elementos no fluxo normal do **HTML** com o resultado obtido ao aplicar a propriedade `display: grid`. Observe que, nesse primeiro momento, os itens continuam organizados em uma única coluna. Isso ocorre porque ainda não definimos explicitamente a quantidade de colunas do contêiner. Para controlar a quantidade de colunas e linhas, precisamos utilizar outras propriedades, como `grid-template-columns` e `grid-template-rows`, que serão apresentadas a seguir.

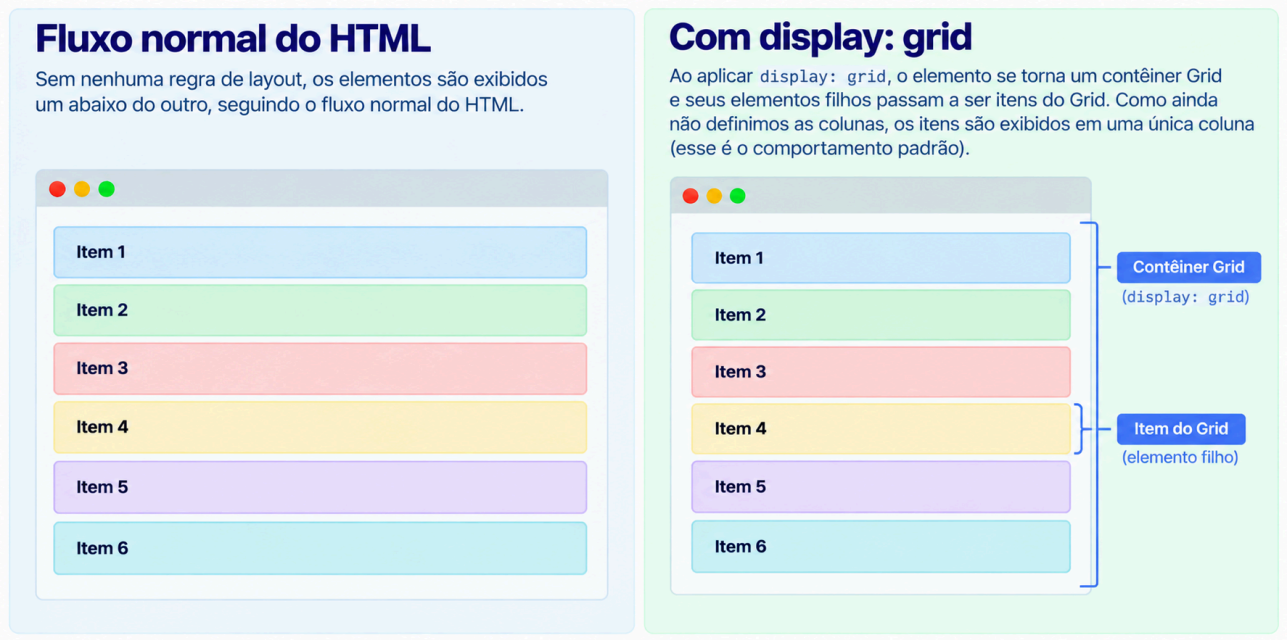


Figura 9 – Comparação entre o fluxo normal do HTML e a aplicação de display: grid.

3.2. Definindo colunas e linhas

A propriedade `grid-template-columns` permite definir a quantidade e a largura das colunas de um contêiner Grid. Com ela, podemos determinar como o espaço disponível será dividido entre os itens. Por exemplo, considere novamente o exemplo com seis elementos **HTML**. Para organizá-los em três colunas, podemos utilizar a seguinte regra **CSS**:

```
CSS
.container {
    display: grid;
    grid-template-columns: 200px 200px 200px;
}
```

Nesse exemplo, a propriedade `grid-template-columns` define três colunas, cada uma com largura de `200px`. Os itens são distribuídos nessas colunas, preenchendo as linhas sucessivamente. Assim, com seis itens, teremos duas linhas com três itens cada, desde que haja espaço suficiente para acomodar as colunas.

Também podemos utilizar unidades diferentes para definir as larguras. Por exemplo, `grid-template-columns: 1fr 2fr 1fr` cria três colunas cujas proporções são 1:2:1, ou seja, a segunda coluna recebe o dobro do espaço destinado a cada uma das outras duas, considerando o espaço disponível para distribuição.

3.2.1. Utilizando a unidade fr

A unidade `fr` representa uma fração do espaço disponível no contêiner Grid. Ela permite distribuir proporcionalmente esse espaço entre as colunas, sem a necessidade de definir larguras fixas em pixels. Isso é especialmente útil quando o espaço disponível varia, como ocorre em dispositivos com diferentes tamanhos de tela, pois as colunas podem ajustar suas dimensões de acordo com o espaço disponível no contêiner.

Por exemplo, no código a seguir, as três colunas dividem igualmente o espaço disponível. A propriedade `gap`, por sua vez, define o espaçamento entre os itens.

```css
.container {
    display: grid;
    grid-template-columns: 1fr 1fr 1fr;
    gap: 16px;
}
```

Entretanto, a utilização da unidade `fr` não garante, por si só, a responsividade do layout, que também depende da estrutura e das demais regras **CSS** utilizadas.

A **Figura 10** apresenta as formas de definir as colunas de um contêiner Grid. No primeiro exemplo, cada coluna possui uma largura fixa de `200px`. No segundo, a unidade `fr` é utilizada para dividir igualmente o espaço disponível entre as três colunas. Já no terceiro, as colunas possuem proporções diferentes, de modo que a segunda recebe o dobro do espaço destinado a cada uma das outras duas. Essa comparação demonstra como podemos controlar a distribuição do espaço no Grid Layout de acordo com as necessidades do layout.

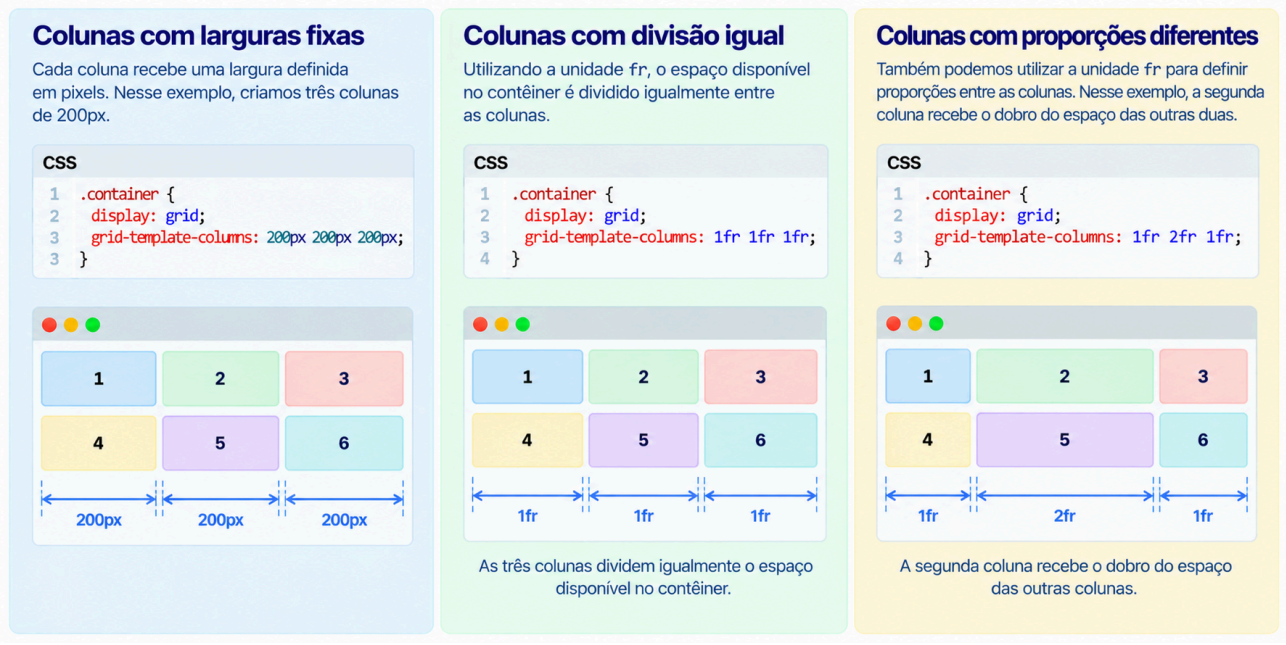


Figura 10 – Comparação entre larguras fixas e distribuição proporcional de colunas no Grid Layout.

3.2.2. Simplificando a definição de colunas com repeat()

Quando precisamos criar várias colunas com a mesma configuração, podemos utilizar a função `repeat()` para simplificar a declaração da propriedade `grid-template-columns`. Por exemplo, a seguinte regra define três colunas de mesma proporção:

```css
.container {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 16px;
}
```

A declaração `repeat(3, 1fr)` equivale a definir três colunas de uma fração cada. Essa forma torna o código mais conciso, especialmente quando precisamos criar grades com várias colunas de mesma largura. A função `repeat()` também pode ser utilizada com valores fixos, como em `repeat(3, 200px)`, que define três colunas com largura de `200px` cada.

É importante lembrar que `repeat()` não define, por si só, um layout responsivo. Ela apenas simplifica a repetição de valores na definição das linhas ou colunas.

Definindo as linhas

A propriedade `grid-template-rows` permite definir as alturas das linhas do Grid. Seu funcionamento é semelhante ao de `grid-template-columns`, mas aplicado à dimensão vertical. Por exemplo:

```css
.container {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    grid-template-rows: 100px 150px;
    gap: 16px;
}
```

Nesse exemplo, definimos três colunas de mesma proporção e duas linhas com alturas de `100px` e `150px`, respectivamente. Como temos seis itens e três colunas, os elementos serão distribuídos em duas linhas.

A **Figura 11** ilustra como as propriedades `grid-template-columns` e `grid-template-rows` permitem controlar as dimensões das colunas e linhas de um

contêiner Grid. No exemplo, a propriedade `grid-template-columns` define três colunas de mesma proporção, utilizando a unidade `fr`, enquanto `grid-template-rows` estabelece duas linhas com alturas diferentes: `80px` e `140px`. Dessa forma, podemos controlar separadamente a distribuição horizontal e vertical dos elementos no layout.

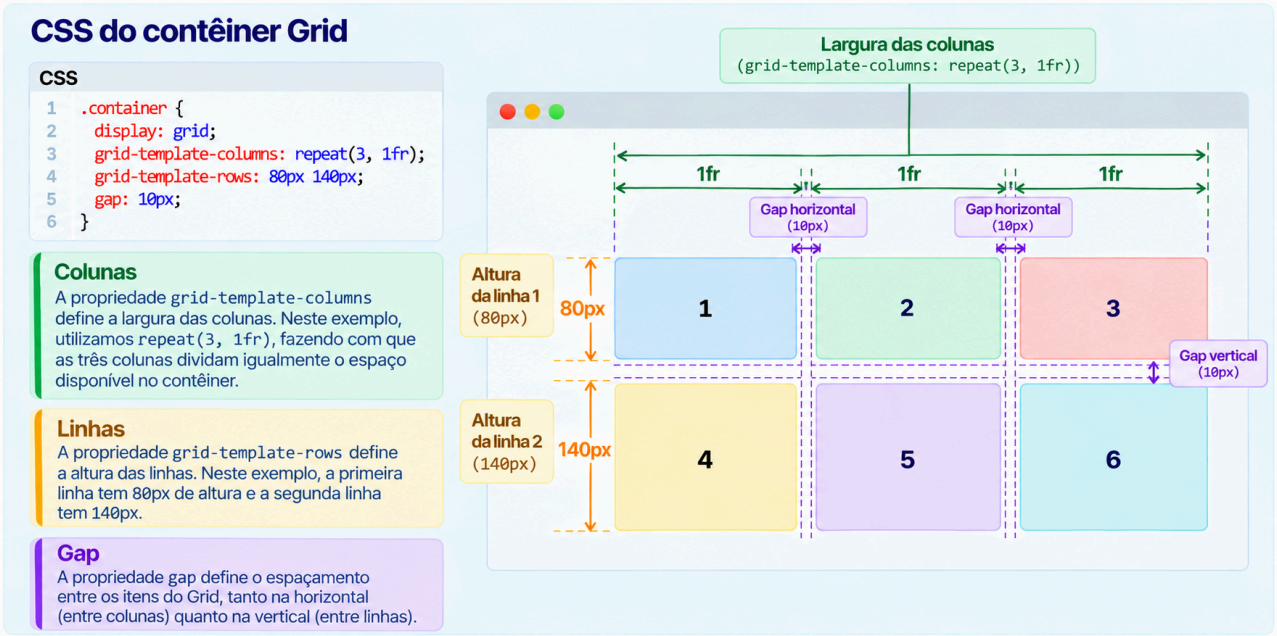


Figura 11 – Definição das dimensões das colunas e linhas com CSS Grid.

Quando não definimos explicitamente todas as linhas necessárias, o navegador cria linhas adicionais automaticamente para acomodar os itens. Suas dimensões são determinadas pelo conteúdo e pelas regras de dimensionamento aplicáveis.

3.3. Posicionando itens no Grid

Além de definir a quantidade e as dimensões das linhas e colunas, o **CSS** Grid permite controlar a posição que cada item ocupa no layout. Dessa forma, podemos fazer com que um elemento ocupe mais de uma coluna ou linha, criando estruturas que vão além de uma grade uniforme.

Para isso, podemos utilizar as propriedades `grid-column` e `grid-row`, que permitem definir as linhas de início e de término de um item no Grid.

3.3.1. Ocupando mais de uma coluna

A propriedade `grid-column` permite definir em quais linhas verticais do Grid um item começa e termina, determinando, consequentemente, quantas colunas ele ocupa. Por exemplo, a regra **CSS** a seguir determina que o item começa na linha vertical 1 e termina na linha vertical 3. Portanto, ele ocupa duas colunas:

```css
.item-destaque {
    grid-column: 1 / 3;
}
```

Também podemos utilizar a palavra-chave `span` para indicar quantas colunas um item deve ocupar:

```css
.item-destaque {
    grid-column: span 2;
}
```

Nesse exemplo, o item ocupa duas colunas, a partir da posição determinada pelo algoritmo de posicionamento automático do Grid, caso nenhuma posição inicial tenha sido especificada.

3.3.2. Ocupando mais de uma linha

De maneira semelhante, a propriedade `grid-row` permite controlar as linhas horizontais que delimitam a posição de um item, possibilitando que ele ocupe mais de uma linha do Grid. Por exemplo, a regra **CSS** a seguir determina que o item começa na linha 1 e termina na linha 3. Portanto, ele ocupa duas linhas.

```css
.item-destaque {
    grid-row: 1 / 3;
}
```

Também podemos utilizar `span` para indicar a quantidade de linhas ocupadas:

```css
.item-destaque {
    grid-row: span 2;
}
```

Assim como ocorre com a propriedade `grid-column`, o item ocupará duas linhas, respeitando o posicionamento definido ou determinado automaticamente pelo Grid.

A **Figura 12** ilustra como as propriedades `grid-column` e `grid-row` permitem que um item ocupe mais de uma coluna ou linha em um contêiner Grid. No exemplo à esquerda, o primeiro item ocupa duas colunas, enquanto, no exemplo à direita, ele ocupa duas linhas.

Esse recurso permite criar layouts mais flexíveis, nos quais determinados elementos podem ocupar áreas maiores que os demais. Por exemplo, em uma galeria de imagens, uma imagem em destaque pode ocupar duas colunas, enquanto as outras permanecem em células individuais.

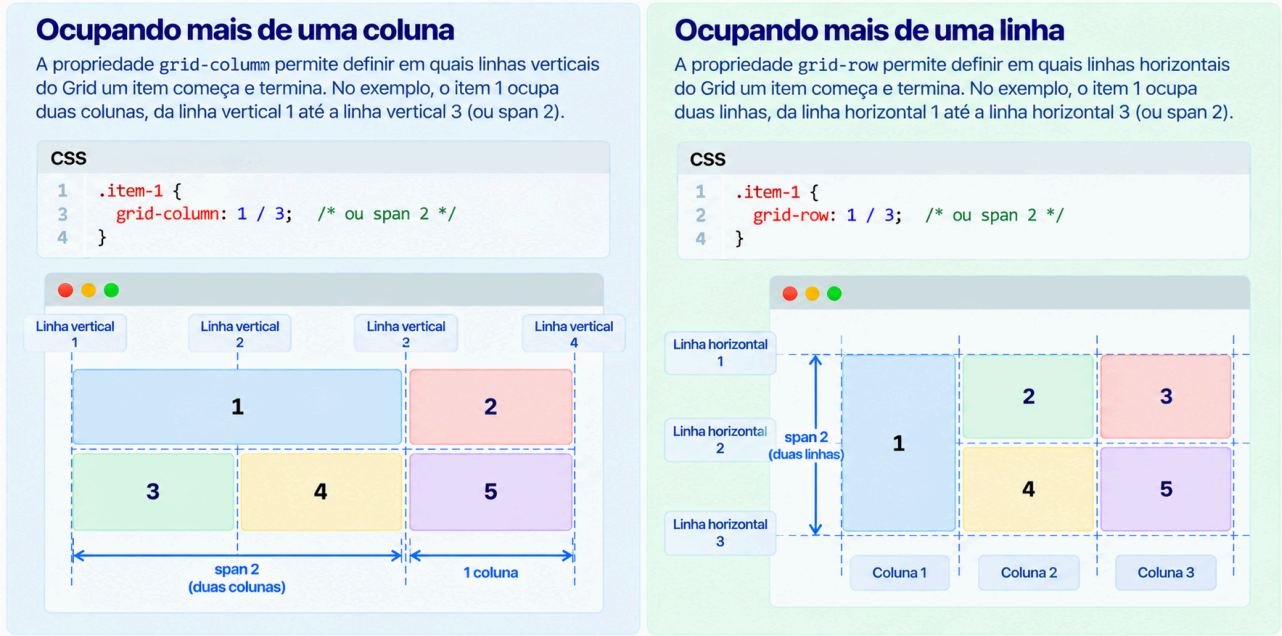


Figura 12 – Posicionamento de itens em múltiplas colunas e linhas com CSS Grid.

4. Aplicando o Grid Layout em uma página Web

O Grid Layout pode ser utilizado para organizar diferentes regiões de uma página **Web**, facilitando a distribuição dos elementos e a construção de layouts visualmente estruturados. Para compreender sua aplicação, vamos considerar uma página composta por cabeçalho, menu lateral, conteúdo principal e rodapé.

Considere a seguinte estrutura **HTML**:

```HTML
<div class="pagina">
    <header>Cabeçalho</header>
    <nav>Menu</nav>
    <main>Conteúdo principal</main>
    <footer>Rodapé</footer>
</div>
```

Podemos utilizar o **CSS** Grid para definir a disposição desses elementos. No exemplo a seguir, o layout possui duas colunas: a primeira tem largura fixa de `200px`, enquanto a segunda utiliza `1fr` para ocupar o espaço disponível. As linhas possuem alturas definidas para o cabeçalho e o rodapé, enquanto a região central recebe o espaço restante, conforme as dimensões disponíveis e o conteúdo.

A propriedade `gap` define o espaçamento entre as regiões. Já as propriedades `grid-column` e `grid-row` permitem posicionar o cabeçalho e o rodapé ao longo das duas colunas, enquanto o menu e o conteúdo principal ocupam regiões distintas na área central. Esse exemplo demonstra como o Grid Layout permite organizar diferentes regiões de uma página por meio de linhas, colunas e posicionamento dos itens.

```css
.pagina {
    display: grid;
    grid-template-columns: 200px 1fr;
    grid-template-rows: 80px 1fr 60px;
    gap: 10px;
    min-height: 400px;
}

header {
    grid-column: 1 / 3;
}

nav {
    grid-column: 1;
    grid-row: 2;
}

main {
    grid-column: 2;
    grid-row: 2;
}

footer {
    grid-column: 1 / 3;
}
```

A **Figura 13** ilustra a aplicação do Grid Layout na construção de uma página Web composta por cabeçalho, menu lateral, conteúdo principal e rodapé. Observe como as propriedades `grid-template-columns` e `grid-template-rows` definem a estrutura do layout, enquanto `grid-column` permite que o cabeçalho e o rodapé ocupem as duas colunas. O exemplo reúne os conceitos apresentados nesta seção e demonstra como o CSS Grid pode ser utilizado para organizar diferentes regiões de uma página.

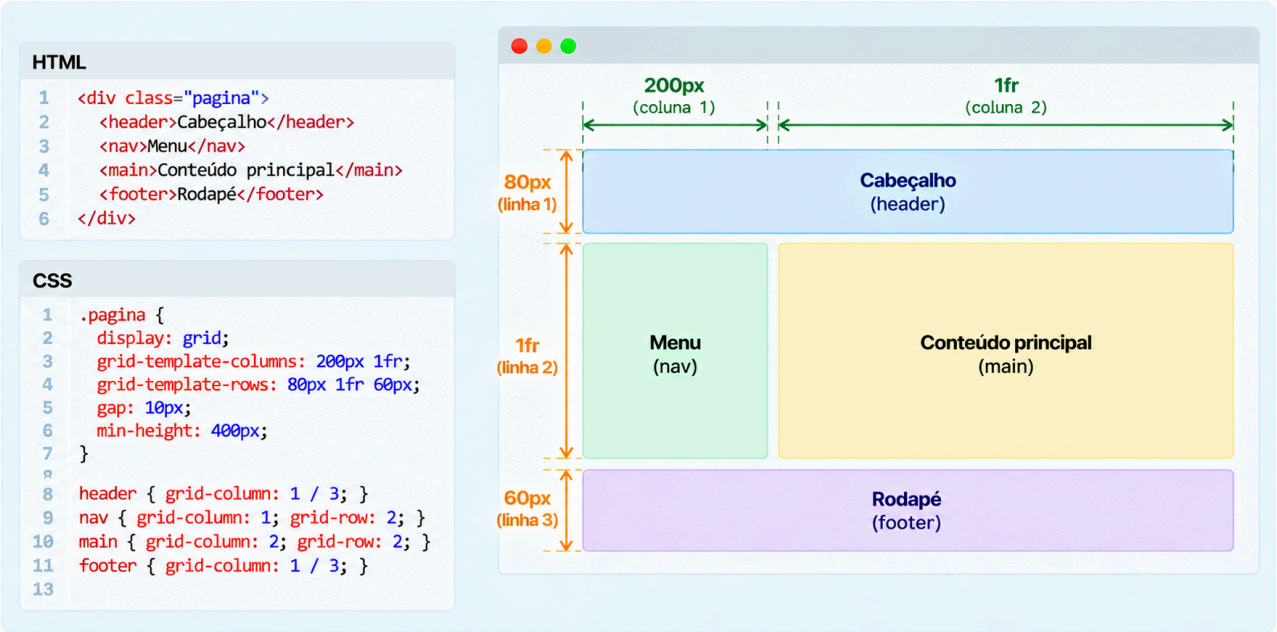


Figura 13 – Aplicação do Grid Layout na organização das regiões de uma página Web.

5. Flexbox ou Grid Layout?

O Flexbox e o Grid Layout são recursos do **CSS** que permitem organizar elementos **HTML** de diferentes maneiras. Embora ambos possam ser utilizados na construção de layouts, cada um apresenta características que o tornam mais adequado a determinadas situações.

O Flexbox é especialmente indicado quando precisamos organizar e distribuir elementos em uma única dimensão, seja em uma linha ou em uma coluna. Ele é útil, por exemplo, para alinhar itens de um menu de navegação, distribuir botões ou organizar elementos de um formulário.

O Grid Layout, por sua vez, é mais indicado quando precisamos controlar simultaneamente linhas e colunas. Ele facilita a construção de estruturas como galerias de imagens, conjuntos de cartões e diferentes regiões de uma página **Web**.

A tabela a seguir resume as principais diferenças entre esses recursos.

Característica	Flexbox	Grid Layout
Dimensão principal	Uma dimensão: linha ou coluna	Duas dimensões: linhas e colunas
Principal objetivo	Alinhar e distribuir itens	Organizar itens em uma grade
Exemplos de uso	Menus, grupos de botões e alinhamentos	Galerias, cartões e regiões da página
Propriedade inicial	`display: flex`	`display: grid`

É importante destacar que esses recursos não são concorrentes nem mutuamente exclusivos. Podemos utilizar Flexbox e Grid Layout na mesma página, escolhendo cada um de acordo com a necessidade de organização de determinada região.

Em resumo: utilize Flexbox quando o foco estiver na distribuição dos elementos em uma linha ou coluna; prefira Grid Layout quando precisar controlar a organização em linhas e colunas simultaneamente. Essa distinção serve como orientação geral, e não como uma regra rígida.

6. Para lembrar

- O **Flexbox** e o **Grid Layout** são recursos do **CSS** utilizados para organizar elementos **HTML** e construir layouts para páginas **Web**.
- A propriedade `display: flex` transforma um elemento em um contêiner Flexbox, cujos filhos diretos são organizados como itens flexíveis.
- O valor `inline-flex` permite criar um contêiner que se comporta como um elemento de nível de linha.
- No Flexbox, a propriedade `flex-direction` define a direção dos itens, enquanto `justify-content` controla sua distribuição ao longo do eixo principal e `align-items` controla seu alinhamento no eixo transversal.
- A propriedade `gap` define o espaçamento entre os itens, enquanto `flex-wrap` permite que eles sejam distribuídos em várias linhas ou colunas quando não há espaço suficiente.
- O **Grid Layout** permite organizar elementos em duas dimensões, controlando simultaneamente linhas e colunas.
- A propriedade `display: grid` transforma um elemento em um contêiner Grid, e seus filhos diretos tornam-se itens do Grid.
- As propriedades `grid-template-columns` e `grid-template-rows` definem as dimensões das colunas e linhas, respectivamente. A unidade `fr` permite distribuir proporcionalmente o espaço disponível, e a função `repeat()` simplifica a repetição de valores.
- No Grid Layout, `gap` define o espaçamento entre linhas e colunas, enquanto `grid-column` e `grid-row` permitem controlar a posição dos itens e fazer com que ocupem múltiplas colunas ou linhas.
- O Flexbox é especialmente adequado à organização de elementos em uma dimensão, enquanto o Grid Layout facilita a construção de estruturas em duas dimensões. Os dois recursos podem ser utilizados em conjunto na mesma página **Web**.